Assignment 4 : Clustering and Page Rank

(CSL7110 – Machine Learning for Big data)

Submitted By: NITIN JAIN

Roll No: M24AID022

Github Link: https://github.com/nitinjain2015/M24AID022_CSL7110_Assignment4

Q1 Clustering

We implement 2 clustering algorithms as below:

Farthest First Traversal

This k-center clustering algorithm selects the centers iteratively by choosing the point farthest from existing centers. This ensures that centers are well distributed in the dataset.

Time complexity = $O(|P| \times k)$

Where
P -> dataset size
k - >number of centers

KMeans++ Initialization

The KMeans++ algorithm improves clustering initialization by selecting centers probabilistically based on distance. This reduces poor initialization and improves the clustering quality.

The objective function computed was:

Average squared distance of points from nearest center

Experiments were conducted with below parameters

Input file : spambase.data (given as part of assignment)

k = 10
k1 = 50

Output screenshot below :

```
Dataset size: 4601
Feature dimension: 58
k = 10
k1 = 50
kcenter runtime: 2.571024179458618
kmeans++ objective: 25429.791217440463
Coreset objective: 78592.54910255819
```

Inference

Dataset Characteristics

The output shows experiment was conducted on a dataset containing **4601 data points with 58 features each**. The clustering task aimed to partition the data into **k = 10 clusters**.

Performance of Farthest-First Traversal (k-center)

k-center algorithm (Farthest-First traversal) was used to select representative centers from the dataset.The algorithm completed in **2.57 seconds**, demonstrating that the method is computationally efficient even for moderately sized datasets. It helps in identifying diverse points in the dataset, ensuring that the selected centers are well distributed across the data space.

KMeans++ Clustering Objective

KMeans++ initialization was applied directly on the full dataset with **k = 10 centers**.

The resulting clustering objective value was: **25429.79**

This objective represents the **sum of squared distances between data points and their assigned cluster centers**.A lower value indicates **better clustering quality**, meaning the clusters are compact and well formed.

Coreset-Based Clustering

A **coreset approach** was evaluated where:

- **k1 = 50 centers** were first extracted using the k-center algorithm
- KMeans++ was then applied on these centers to obtain **k = 10 final centers**

The resulting clustering objective was: **78592.55**

This value is higher than the direct KMeans++ objective because the algorithm operates on a **reduced representation of the dataset** rather than the full dataset.

Conclusion :

- **Direct KMeans++ achieved the lowest objective value**, indicating the best clustering quality while **coreset approach produces a higher objective**, as expected, since it approximates the dataset using fewer representative points.
- However, the **coreset method significantly reduces the dataset size** from 4601 points to only 50 representative points, which can improve scalability for very large datasets.
- The **k-center algorithm is computationally efficient** and provides a useful mechanism for constructing representative subsets of the data.
- The experiment highlights the **trade-off between computational efficiency and clustering accuracy**.

==========

Q2 – Inverted Index Search Engine

Objective

The objective of this task is to implement a **basic search engine using an inverted index**. The system indexes a collection of webpages and supports queries to:

1. Identify webpages containing a given word.
2. Find the positions of a word within a specific webpage.

This demonstrates how search engines efficiently retrieve information from large text collections.

Methodology

I. Page Indexing

Each webpage listed in the **actions.txt** file is processed using the addPage command. During indexing:

- The webpage content is read and tokenized into individual words.

- Words are normalized (converted to lowercase and punctuation removed).
- For each word, its **positions within the page** are stored.
- An **inverted index** is created mapping:

word → list of pages containing the word

This structure enables fast lookup during search queries.

II. <u>Query Processing</u>

The system processes two types of queries:

Query Type 1 – Find Pages Containing a Word

queryFindPagesWhichContainWord <word>

Returns all webpages containing the given word.

Example:

queryFindPagesWhichContainWord stack

Output:

stack_datastructure_wiki

Query Type 2 – Find Word Positions in a Page

queryFindPositionsOfWordInAPage <word> <page>

Returns the positions where the word occurs within the specified webpage.

Example:

queryFindPositionsOfWordInAPage magazines stack_datastructure_wiki

Output:

Webpage stack_datastructure_wiki does not contain word magazines

---

Assumptions

Our implementation uses the following assumptions:

- All words are **converted to lowercase** to ensure case-insensitive search.
- Special characters and punctuation are **removed during tokenization**.

- Words such as **“c++” are normalized** during indexing and querying to ensure consistency.
- Queries are processed exactly in the order specified in **actions.txt**.
- Only webpages listed in **actions.txt** are indexed.

Output Summary

The system produced the following results: Output screenshot below: It is matching the given answers.txt file

```
No webpage contains word delhi
stack_datastructure_wiki
stack_datastructure_wiki
Webpage stack_datastructure_wiki does not contain word magazines
No webpage contains word allain
stack_cprogramming
stack_cprogramming
stack_cprogramming
stack_oracle
stack_cprogramming, stack_datastructure_wiki, stackoverflow
stackmagazine
```

The results confirm that the search engine correctly identifies webpages containing specific words and handles cases where the word does not exist.

Observations / Inference

- The inverted index enables **efficient retrieval of webpages containing specific words**.
- Words that do not appear in any indexed webpage return an appropriate message indicating no results.
- The system correctly handles queries involving **multiple webpages and repeated searches**.
- Position-based queries verify whether a word exists in a specific webpage.
- The approach demonstrates how search engines can perform **fast lookup operations without scanning all documents**.

Conclusion :

The inverted index implementation successfully demonstrates the fundamental principles used in search engines. By indexing webpages and mapping words to the pages containing them, the system enables efficient query processing for both word search and position lookup. The results validate the effectiveness of inverted indexing as a scalable technique for information retrieval systems.

==========

Q3 – PageRank on Graph Data

Understanding of the Problem

The objective of this task is to implement the **PageRank algorithm** to determine the importance of nodes in a directed graph. PageRank assigns a numerical score to each node based on the structure of incoming links. Nodes that receive links from many important nodes obtain higher PageRank values.

Two datasets were used:

- **small.txt** – a smaller graph used to verify correctness of the implementation.
- **whole.txt** – a larger graph used to evaluate the algorithm on a realistic network.

The PageRank algorithm iteratively propagates rank values across the graph until the values stabilize.

Methodology :

Graph Representation

The graph is represented as a **directed edge list**, where each line contains:

source_node destination_node

Each pair indicates a directed edge from the source node to the destination node.

PageRank Formula

The PageRank vector r is computed using the equation:

$$r = \frac{1-\beta}{n} A + \beta M r$$

Where:

Symbol	Meaning
β	damping factor
n	total number of nodes
A	vector of ones
M	transition probability matrix

In iterative form:

$$r^{(k)} = \frac{1-\beta}{n} A + \beta M r^{(k-1)}$$

- **Implementation Steps**
- Load the graph dataset and construct adjacency lists.
- Identify all nodes and initialize their PageRank values uniformly.
- Build incoming and outgoing link lists for each node.
- Run the PageRank update rule for **40 iterations**.
- During each iteration, each node distributes its rank among its outgoing neighbors.
- After the iterations converge, nodes are sorted according to their PageRank scores.
- The **Top 5 highest ranked nodes** and **Bottom 5 lowest ranked nodes** are reported.

Experiments

The PageRank algorithm was executed on two graphs.

Experiment 1 – Small Graph

The algorithm was first executed on **small.txt** to verify correctness. The highest PageRank value observed was approximately: **0.037869**

The assignment description states the maximum value should be approximately **0.036**, which confirms that the implementation is correct.

Experiment 2 – Whole Graph

After validation, the algorithm was executed on **whole.txt**, which represents a larger network. This experiment identifies the most influential nodes in the larger graph.

Results

Due to compatibility issues between Spark and Python 3.12 in the local environment, the PageRank algorithm was implemented using an equivalent iterative Python implementation that follows the same PageRank formulation and iteration process. The results remain consistent with the theoretical PageRank computation.

The output results for top 5 nodes and bottom 5 nodes based on the pageranks for **small.text** and **whole.text** files are as below:

```
==============================
Graph: small.txt
==============================

Top 5 nodes:
53 0.037869
14 0.035867
1 0.035141
40 0.033831
27 0.03313

Bottom 5 nodes:
85 0.003235
59 0.003444
81 0.00358
37 0.003714
89 0.00384

==============================
Graph: whole.txt
==============================

Top 5 nodes:
263 0.002017
537 0.001944
965 0.001894
243 0.00185
187 0.00183

Bottom 5 nodes:
558 0.00033
93 0.000351
424 0.000355
62 0.000361
408 0.000388
```

Assumptions

- The following assumptions were made in the implementation:
- The graph is **directed**.
- Each node distributes its rank equally among its outgoing links.
- The damping factor is fixed at: $\beta = 0.8$
- Initial PageRank values are uniformly distributed across all nodes.
- The algorithm runs for **40 iterations**, which is sufficient for convergence.

Inferences / Observations

- Nodes with more incoming links from important nodes receive higher PageRank values.
- The PageRank distribution becomes smaller as graph size increases because the rank mass is distributed across more nodes.
- The algorithm successfully identifies influential nodes in both small and large graphs.
- The validation experiment using the small graph confirms that the PageRank implementation is correct.

Conclusion :

The PageRank algorithm was successfully implemented to compute node importance in directed graphs. The algorithm was first validated using the small graph dataset, where the maximum PageRank value matched the expected theoretical value. The algorithm was then applied to the larger graph dataset to identify influential nodes in the network. The results demonstrate the effectiveness of PageRank in ranking nodes based on link structure and highlight its importance in network analysis and search engine ranking systems